# Neuro-Adaptive Control Without State Measurement: A 1D-CNN Framework for Higher-Order Systems Using Historical Output Data

Myeongseok Ryu, Naol Samuel Erega, and Kyunghwan Choi

***Abstract*—This study proposes a convolutional neural network (CNN)-based adaptive controller with three notable features: 1) it determines control input directly from historical sensor data; 2) it learns the desired control policy during real-time implementation without using a pre-trained network; and 3) the asymptotic tracking error convergence is proven during the learning process. An adaptive law for learning the desired control policy is derived using the gradient descent optimization method, and its stability is analyzed based on the Lyapunov approach. A simulation study using a control-affine nonlinear system demonstrated that the proposed controller exhibits these features, and its performance can be tuned by manipulating the design parameters. In addition, it is shown that the proposed controller has a superior tracking performance to that of a deep neural network (DNN)-based adaptive controller.**

***Index Terms*—Neuro-adaptive control, convolutional neural network (CNN), ...**

## NOTATION

In this study, the following notation is used:

- $\mathbb{R}^n$ denotes the $n$-dimensional Euclidean space.
- $\mathbb{R}^{n\times m}$ denotes the set of $n \times m$ real matrices.
- A vector and a matrix are denoted by $\boldsymbol{x} = [x_i] \in \mathbb{R}^n$ and $\boldsymbol{A} := [a_{ij}] \in \mathbb{R}^{n\times m}$, respectively.
- $\boldsymbol{I}_n$ denotes the $n \times n$ identity matrix, and $\boldsymbol{0}_{n\times m}$ denotes the $n \times m$ zero matrix.
- $\mathrm{vec}(\boldsymbol{A})$ denotes the vectorization of the matrix $\boldsymbol{A}$.
- A matrix $\boldsymbol{P} \succ 0$ ($\boldsymbol{P} \succeq 0$) is positive definite (positive semidefinite).
- $\lambda_{\min}(\boldsymbol{A})$ denotes the minimum eigenvalue of the matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$.
- $\otimes$ denotes the Kronecker product [1].

## I. INTRODUCTION

### A. Motivation

Neural networks (NNs) have been widely used in control applications as a function approximator for adaptive control [2], state estimation [3], and so on. These studies provided the ultimate boundedness of tracking or estimation errors based on the Lyapunov-based stability analysis but were limited to

Deep neural networks (DNNs) with multiple hidden layers are more expressive and can provide better performance than shallow NNs. However, due to the difficulty in deriving an adaptation law ensuring stability, designing a stable DNN-based controller is generally considered challenging. There have been a few attempts at overcoming this challenge, including developing Lyapunov-based adaptation laws for controllers based on DNNs or their variations [4]–[6]. A Lyapunov-based adaptation law for a DNN-based controller was derived in [4], which ensured asymptotic convergence of tracking error. [5] utilized long short-term memory (LSTM), a type of DNN, in designing an adaptive controller and presented the ultimate boundedness of tracking error based on Lyapunov analysis. The Lyapunov-based approach was extended to using physics-informed LSTM for an adaptive controller and therein demonstrated its asymptotic stability [6].

While DNN-based adaptive control with a stability guarantee has been studied by pioneering works [4]–[6], convolutional neural networks (CNNs), which are well known for their spatial feature capturing ability, have not been as actively investigated in control applications, as they have been in computer vision applications. Nonetheless, motivated by the feature-capturing capability, the use of CNNs as a basis for end-to-end controllers has been studied [7]–[10]. These end-to-end controllers determine the control input directly from the sensor data with features that are extracted by CNNs. In [7], [8], CNN-based end-to-end controllers were trained offline to produce the desired steering behavior based on 2D images from the camera. [9], [10] generated pseudo-2D images by stacking historical system input or output data and used them to train CNN-based end-to-end controllers offline to behave as target controllers. However, none of the works above considered online adaptation or stability analysis of CNN-based end-to-end controllers.

This study presents a CNN-based adaptive controller with a stability guarantee. The proposed controller uses 2D images generated by stacking historical sensory data as an input matrix to convolutional layers (CVLs). The output of CVLs is input to the following fully connected layers (FCLs), which produce the controller output. An adaptation law of network weights is implemented to learn the desired control policy and is derived using the gradient descent optimization method. The stability of the adaptation laws is proven based on the Lyapunov analysis by showing that the tracking error

Myeongseok Ryu, Naol Samuel Erega and Kyunghwan Choi are with the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34051, Korea (e-mail: dding_98@kaist.ac.kr, samuelnaol7@kaist.ac.kr and kh.choi@kaist.ac.kr).

[MS: Should be reviewed] This work was supported in part by a Korea Research Institute for Defense Technology planning and advancement (KRIT) grant funded by Korea government DAPA (Defense Acquisition Program Administration) (No. KRIT-CT-22-087, Synthetic Battlefield Environment based Integrated Combat Training Platform) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025- 00554087). (Corresponding author: Kyunghwan Choi)

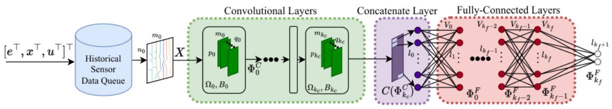


Fig. 1: System Architecture.

is asymptotically convergent and the network weights and biases are bounded. A part of the proposed controller is designed based on these findings, such as Jacobians with respect to FCLs from [4] and stabilizing techniques for the adaptation law from [2], [3]; however, the remainder of the proposed controller, particularly the components rely on the mathematical formulation and adaptation law of the overall CNN architecture, has been solely developed by the present authors. A simulation study using a control-affine nonlinear system is performed to demonstrate that the proposed CNN-based controller ensures asymptotic convergence of tracking errors during online adaptation without using a pre-trained network. In addition, the tracking performance of the proposed controller is compared with that of the DNN-based adaptive controller, which is a form in which the CVLs are not included in the proposed controller.

### B. Contributions

### C. Organization

The remainder of this paper is organized as follows. Section II describes the system details of the proposed CVL-CONAC architecture. Section III presents the problem formulation and control objectives. Section IV develops the adaptation laws. Section V provides the stability analysis. Section VI presents the numerical validation results. Finally, Section VII concludes the paper.

## II. System Details

The proposed controller architecture is shown in Fig. 1. The network $\widehat{\mathbf{\Phi}}$ consists of CVLs and FCLs, which are detailed in the following subsections.

### A. CVL-CONAC Architecture

The network $\widehat{\mathbf{\Phi}}$ is designed as a hybrid architecture suitable for capturing temporal dynamics from historical data[??].

*1) Convolutional Layers (CVLs):* The CVLs are represented recursively as:

$$\Phi^C_{j_c} = \begin{cases} O(\phi^C_{j_c}(\Phi^C_{j_c-1}), \Omega_{j_c}, B_{j_c}), & j_c \in [1, \dots, k_c] \\ O(\phi^C_0(X), \Omega_0, B_0), & j_c = 0 \end{cases}, \quad (1)$$

where $X \in \mathbb{R}^{n_0 \times m_0}$ denotes the network input matrix, $O : \mathbb{R}^{n_{j_c} \times m_{j_c}} \times \mathbb{R}^{p_{j_c} \times m_{j_c} \times q_{j_c}} \times \mathbb{R}^{q_{j_c}} \to \mathbb{R}^{n_{j_c+1} \times m_{j_c+1}}$ denotes the CNN operator (see Appendix A), and $\phi^C_{j_c} : \mathbb{R}^{n_{j_c} \times m_{j_c}} \to \mathbb{R}^{n_{j_c} \times m_{j_c}}$ denotes the matrix activation function (i.e., $\phi^C(\Phi^C_{j_c-1})_{(i,j)} = \sigma_{j_c}(\Phi^C_{j_c-1})_{(i,j)}$) for some activation function $\sigma_{j_c} : \mathbb{R} \to \mathbb{R}$. The first activation function $\phi^C_0$ should be a bounded nonlinear function to ensure that the input to the first CNN operator is bounded. In this study, $\alpha_1 \tanh(\cdot)$ is selected with $\alpha_1 \in \mathbb{R}_{>0}$. The filter set $\Omega_{j_c}$ contains $q_{j_c}$ filters $W^{(i)}_{j_c} \in \mathbb{R}^{p_{j_c} \times m_{j_c}}$, $\forall i \in [1, \dots, q_{j_c}]$, and is represented as $\Omega_{j_c} = \{W^{(1)}_{j_c}, \dots, W^{(q_{j_c})}_{j_c}\}$, where superscript $i$ denotes the filter index. The bias vector $B_{j_c}$ consists of $q_{j_c}$ biases.

The weights for the $j_c$-th CVL layer are collectively represented as $\hat{\theta}_{Cj_c} \triangleq [\mathrm{vec}(\Omega_{j_c})^T, B^T_{j_c}]^T$.

*2) Fully-Connected Layers (FCLs):* The output matrix of the CVLs is input to the concatenate layer $C(\Phi^C_{k_c}) = [\mathrm{vec}(\Phi^{C^\top}_{k_c})^\top, 1]^\top$ before the input layer of FCLs for compatibility between the CVLs and FCLs. The FCLs are represented recursively as:

$$\Phi^F_{j_f} = \begin{cases} V^\top_{j_f} \phi^F_{j_f}(\Phi^F_{j_f-1}), & j_f \in [1, \dots, k_f] \\ V^\top_0 C(\Phi^C_{k_c}), & j_f = 0 \end{cases}, \quad (2)$$

where $V_{j_f} \in \mathbb{R}^{l_{j_f}+1 \times l_{j_f+1}}$ denotes the weight matrix, $\phi^F_{j_f} : \mathbb{R}^{l_{j_f}} \to \mathbb{R}^{l_{j_f}+1}$ denotes the vector activation function defined by $\phi^F_{j_f}(\Phi^F_{j_f-1}) \triangleq [\sigma_{j_f}(\Phi^F_{j_f-1})^\top, 1]^\top$ for $j_f \in [1, \dots, k_f - 1]$, for some nonlinear activation functions $\sigma_{j_f} : \mathbb{R} \to \mathbb{R}$ such as $\tanh(\cdot)$. Note that $C(\Phi^C_{k_c})$ and $\Phi^F_{j_f}$ are augmented by 1 to consider the bias of the preceding input layer as a weight in the weight matrix. The output of the FCLs is the final output and is represented as $\widehat{\mathbf{\Phi}} \triangleq \Phi^F_{k_f}$.

The FCL weights for the $j_f$-th layer are $\hat{\theta}_{Fj_f} \triangleq \mathrm{vec}(V_{j_f})$.

## III. Problem Formulation and Control Objectives

### A. System Dynamics

The following four wheel steered(4WS) 2-DOF vehicle lateral dynamics [11] are considered:

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t} v_y = \frac{1}{m}(F_f + F_r) - v_x r, \\ \frac{\mathrm{d}}{\mathrm{d}t} r = \frac{1}{I_z}(F_f l_f - F_r l_r), \end{cases} \quad (3)$$

where $v_x$ is the longitudinal velocity, $v_y$ is the lateral velocity, and $r = \frac{\mathrm{d}}{\mathrm{d}t}\psi$ is the yaw rate, $m$ is the vehicle mass and $I_z$ is the yaw moment of inertia, $l_r$ and $l_r$ are the distances from the center of gravity to the front and rear axles, respectively, and $F_f$ and $F_r$ are the front and rear lateral tire forces, respectively, and modelled as:

$$F_i = F_{\max,i} \tanh\left(\frac{C_i}{F_{\max,i}} \alpha_i\right), \quad \forall i \in \{f, r\}, \quad (4)$$

where $C_i$, $F_{\max,i}$, and $\alpha_i$ are the cornering stiffness, saturation limit, and slip angle of the front ($i = f$) and rear ($i = r$) tires, respectively. The slip angles are given by

$$\alpha_f = \delta_f - \frac{v_y + l_f r}{v_x}, \quad \alpha_r = \delta_r - \frac{v_y - l_r r}{v_x}, \quad (5)$$

where $\delta_f$ and $\delta_r$ are the effective front and rear steering angles respectively. In order to capture the lag between the steering commands and the actual steering angles, the actuator dynamics are modeled as: $\frac{\mathrm{d}}{\mathrm{d}t}\delta_i = \frac{\omega_i - \delta_i}{\tau}$, $\forall i \in \{f, r\}$, where $\omega_i$, $\forall i \in \{f, r\}$ denotes the front and rear steering commands, respectively, and $\tau > 0$ is the actuator time constant.

The global position kinematics are given by (6):

$$\dot{X}_a = v_x \cos\psi - v_y \sin\psi, \quad \dot{Y}_a = v_x \sin\psi + v_y \cos\psi, \quad (6)$$

where $(X_a, Y_a)$ denotes the vehicle position in the global frame, and $\psi$ denotes the yaw angle. The yaw rate $\dot{\psi}$ is obtained by integrating the yaw acceleration $\ddot{\psi}$ in (3).

## B. Control Objectives

An end to end controller to is designed to drive errors defined by (7) to zero:

$$e = \begin{bmatrix} e_y \\ e_\psi \end{bmatrix} = \begin{bmatrix} y - y_{\text{des}} \\ \psi - \psi_{\text{des}} \end{bmatrix}, \tag{7}$$

which is obtained by solving optimization problem in (8):

$$\begin{aligned} \min_{\hat{\theta}} \quad & \mathcal{J}(e,\hat{\theta}) = \tfrac{1}{2} e^\top e \\ \text{s.t.} \quad & c_i(\hat{\theta}) = \tfrac{1}{2}\left(\|\hat{\theta}_i\|^2 - \bar{\theta}_i^2\right) \le 0, \quad \forall i \in \mathcal{I} \\ & c_{\widehat{\mathbf{\Phi}}}(\hat{\theta}) = \tfrac{1}{2}\left(\|\widehat{\mathbf{\Phi}}\|^2 - \bar{u}^2\right) \le 0, \end{aligned} \tag{8}$$

where

$$\mathcal{I} = \{C0, C1, \ldots, Ck_c\} \cup \{F0, F1, \ldots, Fk_f\},$$

$c_i$ are the weight norm constraints (i.e. ensuring boundedness) and $c_{\widehat{\mathbf{\Phi}}}$ is the output norm constraint (i.e. ensuring input saturation limits $\bar{u}$).

# IV. Adaptation Laws

Solving the optimization problem in (8) is equivalent to finding the saddle point of the Lagrangian function given in (9):

$$\mathcal{L}(\hat{\theta}, \boldsymbol{\lambda}) = \tfrac{1}{2} e^\top e + \sum_{i \in \mathcal{I}} \lambda_i c_i(\hat{\theta}) + \lambda_{\widehat{\mathbf{\Phi}}} c_{\widehat{\mathbf{\Phi}}}(\hat{\theta}), \tag{9}$$

where $\lambda_i, \lambda_{\widehat{\mathbf{\Phi}}} \ge 0$ are the Lagrange multipliers.

## A. Primal Update

The weight update for each block $\hat{\theta}_i$ is proportional to the negative gradient of the Lagrangian: $\dot{\hat{\theta}}_i = -\alpha \nabla_{\hat{\theta}_i} \mathcal{L}$.

*1) Objective Gradient:* The objective gradient is approximated using the chain rule and the sensitivity approximation $\frac{\partial x}{\partial u} \approx \mathbf{I_n}$:

$$\nabla_{\hat{\theta}_i} \mathcal{J} = \left(\frac{\partial e}{\partial \hat{\theta}_i}\right)^T e \approx \left(\frac{\partial x}{\partial u}\frac{\partial u}{\partial \hat{\theta}_i}\right)^T e = -\left(\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \hat{\theta}_i}\right)^T e, \tag{10}$$

*2) Constraint Gradients:* The gradients of the constraints are given by:

$$\begin{cases} \nabla_{\hat{\theta}_i} c_i = \hat{\theta}_i, & \forall i \in \mathcal{I} \\ \nabla_{\hat{\theta}_i} c_{\widehat{\mathbf{\Phi}}} = \left(\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \hat{\theta}_i}\right)^T \widehat{\mathbf{\Phi}}, & \forall i \in \mathcal{I}, \end{cases} \tag{11}$$

The resulting weight update rule for the primal variables is:

$$\dot{\hat{\theta}}_i = \alpha \left[ \left(\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \hat{\theta}_i}\right)^T e - \lambda_i \hat{\theta}_i - \lambda_{\widehat{\mathbf{\Phi}}} \left(\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \hat{\theta}_i}\right)^T \widehat{\mathbf{\Phi}} \right], \tag{12}$$

where $\alpha \in \mathbb{R}_{>0}$ is the learning rate.

## B. Dual Update

The Lagrange multipliers are updated using projected gradient ascent:

$$\begin{cases} \dot{\lambda}_j = \beta_j c_j(\hat{\theta}), & \forall j \in \mathcal{I} \cup \{\widehat{\mathbf{\Phi}}\} \\ \lambda_j & = \max(\lambda_j, 0), \end{cases} \tag{13}$$

where $\beta_j \in \mathbb{R}_{>0}$ is the update rate for the corresponding multiplier.

## C. Network Jacobians

The adaptation law in (12) requires weights Jacobians and are given by:

*1) FCL Jacobians:* The Jacobians of $\widehat{\mathbf{\Phi}}$ with respect to the weights of FCL are derived recursively as the follows:

$$\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \hat{\theta}_{Fj_f}} = \begin{cases} \left(\prod_{l=1}^{k_f} V_l^\top \phi_l^{F'}\right) (I_{l_0} \otimes C(\Phi_{k_c}^C)), & j_f = 0 \\ \left(\prod_{l=j_f+1}^{k_f} V_l^\top \phi_l^{F'}\right) (I_{l_{j_f}} \otimes \Phi_{j_f-1}^{F\top}), & j_f \in [1, ;k_f], \end{cases} \tag{14}$$

where $\phi_{j_f}^{F'} \triangleq \frac{\partial \phi_{j_f}^F(x)}{\partial x}$ is the Jacobian of the activation function with respect to some vector $x$.

*2) CVL Jacobians:* The Jacobians of $\widehat{\mathbf{\Phi}}$ with respect to the weights of CVLs (filters $\Omega_{j_c}$ and biases $B_{j_c}$) are given as follows:

$$\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial W_{j_c}^{l_k}} = \sum_{l_i=1}^{n_{j_c+1}} \left( \frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial \Phi_{j_c}^C(l_i, l_k)} \operatorname{row}(l_b : l_e)(\phi_{j_c}^C) \right), \tag{15}$$

$$\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial B_{j_c}(l_k)} = \sum_{l_i=1}^{n_{j_c+1}} \left( \frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial \Phi_{j_c}^C(l_i, l_k)} \cdot 1 \right), \tag{16}$$

with $l_b \triangleq l_i$, $l_e \triangleq l_i + p_{j_c} - 1$, $j_c \in [0, \ldots, k_c]$, $l_k \in [1, \ldots, q_{j_c}]$

where $\Phi_i$ denote the $i$-th output of $\widehat{\mathbf{\Phi}}$, $\Phi_{j_c}^C \triangleq \phi_{j_c}^C(\Phi_{j_c-1}^C)$ for $j_c \in [1, \ldots, k_c]$, $\Phi_0^C \triangleq \phi_0^C(X)$, and $\partial \widehat{\mathbf{\Phi}}_i / \partial \Phi_{j_c}^C$ denotes the backpropagated gradient of $\widehat{\mathbf{\Phi}}_i$ with respect to $\Phi_{j_c}^C$ (obtained using the back-propagation method, detailed in Appendix B).

# V. Stability Analysis

# VI. Numerical Validation

## A. Validation Setup

In this section, the proposed controller is validated through numerical simulation of non-linear vehicle model in (3). The vehicle parameters are selected as $m = 1463$ kg, $I_z = 1967.8\ \text{kg} \cdot \text{m}^2$, $l_f = 1.2$ m, and $l_r = 1.6$ m. The longitudinal velocity is fixed at $v_x = 110$ km/h. The front and rear cornering stiffness values are allowed to vary within $C_i \in [78.4,\ 145.6]$ kN/rad for $i \in \{f, r\}$, which corresponds to a $\pm 30\%$ variation about the nominal value 112 kN/rad.

The desired trajectory is selected as a closed stadium-shaped path composed of two straight segments and two semicircular arcs. Let $L$ and $R$ denote the straight length and turning radius, respectively. Then, the lower and upper straight segments are defined by

$$\mathcal{P}_1 = \{(x, y) \,|\, 0 \le x \le L,\ y = 0\}, \tag{17}$$

$$\mathcal{P}_2 = \{(x, y) \,|\, 0 \le x \le L,\ y = 2R\}, \tag{18}$$

The left and right turning sections are semicircles centered at $C_{\text{L}} = (0, R)$ and $C_{\text{R}} = (L, R)$, respectively. This path contains both zero-curvature and constant-curvature segments and is therefore suitable for evaluating straight-line regulation and turning performance within a single experiment. Figure 2 illustrates the stadium path together with the vehicle pose, body-fixed frame, and tracking error used in the validation setup.

To generate the path-following reference at each simulation step, the current vehicle position $(X_a, Y_a)$ is projected onto

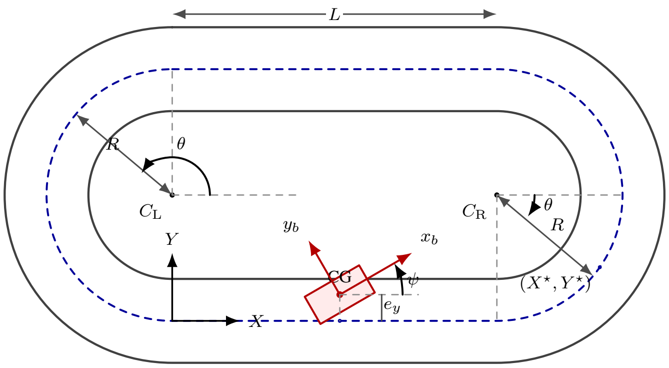


Fig. 2: Stadium-track geometry and vehicle pose used in the validation setup.

the corresponding path segment. For straight segments, the desired heading is:

$$\psi_{\text{des}} = \begin{cases} 0, & \text{lower straight,} \\ \pi, & \text{upper straight,} \end{cases} \tag{19}$$

For the arc segments, the projected point is computed by

$$X^\star = x_c + R\cos\theta, \qquad Y^\star = y_c + R\sin\theta, \tag{20}$$

with

$$\theta = \operatorname{atan2}(Y_a - y_c, X_a - x_c), \tag{21}$$

The desired heading is then taken as the tangent direction of the arc,

$$\psi_{\text{des}} = \theta + \frac{\pi}{2}, \tag{22}$$

The tracking errors are then defined by the lateral displacement from the projected point and the heading error.

### *B. Validation Results*

The validation results are presented in terms of tracking errors, steering commands, and the global vehicle trajectory. In particular, the lateral and heading errors are used to assess whether the proposed 1D CNN–MLP controller can regulate the vehicle toward the desired path while adapting its parameters online. The front and rear steering-rate commands illustrate the control effort generated by the end-to-end policy, and the global $XY$ trajectory provides a direct geometric comparison between the desired path and the actual motion.

In the final manuscript, this subsection should be organized around the simulation plots produced by the current MATLAB implementation. A compact presentation consistent with the reference paper would first discuss the validation conditions, then summarize the tracking behavior in the time domain, and finally comment on the overall path-following accuracy and adaptation behavior. Root-mean-square tracking errors and selected controller settings can also be reported in a table to make the comparison more concise.

## VII. Conclusion

## References

[1] D. S. Bernstein and D. S. Bernstein, *Matrix Mathematics: Theory, Facts, and Formulas (Second Edition)*. Princeton: Princeton University Press, 2009.

[2] K. Esfandiari, F. Abdollahi, and H. A. Talebi, *Neural network-based adaptive control of uncertain nonlinear systems*. Springer, 2022.

[3] H. Talebi, F. Abdollahi, R. Patel, and K. Khorasani, *Neural Network-Based State Estimation of Nonlinear Systems: Application to Fault Detection and Isolation*, ser. Lecture Notes in Control and Information Sciences. Springer New York, 2010.

[4] O. S. Patil, D. M. Le, M. L. Greene, and W. E. Dixon, “Lyapunov-derived control and adaptive update laws for inner and outer layer weights of a deep neural network,” *IEEE Control Systems Letters*, vol. 6, pp. 1855–1860, 2021.

[5] E. J. Griffis, O. S. Patil, Z. I. Bell, and W. E. Dixon, “Lyapunov-based long short-term memory (Lb-LSTM) neural network-based control,” *IEEE Control Systems Letters*, 2023.

[6] R. G. Hart, E. J. Griffis, O. S. Patil, and W. E. Dixon, “Lyapunov-based physics-informed long short-term memory (LSTM) neural network-based adaptive control,” *IEEE Control Systems Letters*, 2023.

[7] V. Rausch, A. Hansen, E. Solowjow, C. Liu, E. Kreuzer, and J. K. Hedrick, “Learning a deep neural net policy for end-to-end control of autonomous vehicles,” in *2017 American Control Conference (ACC)*. IEEE, 2017, Conference Proceedings, pp. 4914–4919.

[8] J. Jhung, I. Bae, J. Moon, T. Kim, J. Kim, and S. Kim, “End-to-end steering controller with cnn-based closed-loop feedback for autonomous vehicles,” in *2018 IEEE intelligent vehicles symposium (IV)*. IEEE, 2018, Conference Proceedings, pp. 617–622.

[9] X. Liu and Y. Dong, “A convolutional neural networks approach to devise controller,” in *MATEC Web of Conferences*, vol. 139. EDP Sciences, 2017, Conference Proceedings, p. 00168.

[10] H. Nobahari and Y. Seifouripour, “A nonlinear controller based on the convolutional neural networks,” in *2019 7th International Conference on Robotics and Mechatronics (ICRoM)*. IEEE, 2019, Conference Proceedings, pp. 362–367.

[11] R. Rajamani, *Vehicle dynamics and control*, 2nd ed., ser. Mechanical Engineering Series. New York, NY: Springer, Dec. 2012.

**Myeongseok Ryu** received the B.S. degree in Mechanical Engineering from Incheon National University, South Korea, in 2023, and the M.S. degree in Mechanical Engineering from GIST, South Korea, in 2025. He is currently pursuing the Ph.D. degree in the Cho Chun Shik Graduate School of Mobility at Korea Advanced Institute of Science and Technology (KAIST), South Korea. His research interests include neuro-adaptive control, online optimization, and contraction theory.

**Naol Samuel Erega** received his B.S. degree in Aerospace Engineering and Electrical Engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2026. He is currently pursuing the M.S. degree at the Cho Chun Shik Graduate School of Mobility, KAIST. His research interests include neural network-based control, system estimations, and unmanned ground vehicle(UGV) dynamics.

**Kyunghwan Choi** received his B.S., M.S., and Ph.D. degrees in Mechanical Engineering from KAIST, Daejeon, South Korea, in 2014, 2016, and 2020, respectively. Following his Ph.D., he worked at the Center for Eco-Friendly & Smart Vehicles at KAIST as a Postdoctoral Fellow and later as a Research Assistant Professor. In 2022, he joined GIST as an Assistant Professor in the Department of Mechanical and Robotics Engineering. Currently, he serves as an Assistant Professor at the Cho Chun Shik Graduate School of Mobility at KAIST, where he is also the Director of the Mobility Intelligence and Control Laboratory. His research focuses on optimal and learning-based control for connected, automated, and electrified vehicles (CAEVs).